IHINGAAI

Agricultural Decision-Support Application

PROJECT DOCUMENTATION

NISR Hackathon Competition 2026

Agricultural Productivity Track

Team: **Pioneers**

30 October 2026

Repository: https://github.com/Aime-Gentil/IhingaAI

Demonstration application: https://ihingaai-pioneers.streamlit.app

Document scope: system design, data and models, functionality, technical implementation, testing, limitations, and usage.

Table of Contents

Acknowledgements 3
1. Executive Summary 4
2. Problem and Proposed Solution 4
2.1 Problem statement 4
2.2 Project objectives 4
2.3 Target users 4
3. Data Sources and Preparation 5
3.1 Prepared datasets and model artifacts 5
3.2 Data workflow 5
3.3 Data governance 5
4. Predictive Analytics Methodology 6
4.1 Estimation approach 6
4.2 Model outputs and safeguards 6
4.3 Evidence and evaluation 6
4.4 Responsible use 6
5. Application Features and User Workflows 7
5.1 Farmer workspace 7
5.2 Agronomist workspace 7
5.3 Planner workspace 7
5.4 End-to-end flow 7
6. System Architecture and Technology 8
6.1 Repository structure 8
6.2 Deployment architecture 8
6.3 Access and secrets 8
7. Testing, Results and Limitations 9
7.1 Testing strategy 9
7.2 Demonstrated outputs 9
7.3 Known limitations 9
7.4 Next development priorities 9
8. Installation, Demonstration and Submission Access 10
8.1 Run locally 10
8.2 Access links 10
9. Conclusion 10
10. References and Project Resources 11

Acknowledgements

We, Team Pioneers, would like to express our sincere gratitude to the National Institute of Statistics of Rwanda (NISR) for organizing the 2026 Hackathon Competition and providing an opportunity to develop innovative, data-driven solutions to real-world challenges.

We appreciate NISR's efforts in promoting the use of statistical data and technology to address important challenges in Rwanda, particularly in the agricultural sector. Access to agricultural survey information provided a valuable foundation for developing the machine-learning component of IhingaAI.

Finally, we acknowledge the importance of Rwanda's agricultural community, whose challenges inspired us to develop a solution that supports informed farming decisions, improves access to agricultural information, and promotes data-driven agricultural productivity.

1. Executive Summary

IhingaAI is a Streamlit-based agricultural decision-support prototype designed for farmers, agronomists and agricultural planners in Rwanda. It connects historical survey-based harvest estimation with everyday field records, farming activities, crop monitoring, weather context, market information and advisory support in one application. The product is intended to improve access to practical information and support decisions, rather than replace field experts or official agricultural statistics.

The application uses two frozen predictive-model artifacts trained on prepared data derived from the 2025 Seasonal Agricultural Survey (SAS). It also provides role-specific workspaces, field-level records and optional generative AI support. Estimates are experimental and uncertain; they are not guaranteed yields or validated current-season forecasts.

2. Problem and Proposed Solution

2.1 Problem statement

Smallholder farmers often need to make crop and resource decisions with fragmented information about production potential, weather, planned activities and spending. Agronomists need a practical way to follow up on assigned farmers, while planners benefit from accessible, appropriately labeled operational data and historical summaries.

2.2 Project objectives

- Provide crop and season-specific production estimates with uncertainty information and suitability checks.
- Allow farmers to register and manage fields, activities, monitoring observations and financial transactions.
- Support permission-controlled engagement between farmers and agronomists.
- Present district-level operational views and contextual information for planning without confusing application records with official statistics.
- Offer understandable AI-assisted explanations while retaining human oversight and clear uncertainty warnings.

2.3 Target users

User	Primary purpose
Farmer	Manage fields, estimate production, monitor crop conditions and plan activities.
Agronomist	Support assigned farmers, review shared field information and provide advice.
Planner	Explore district-level views, assignments and resource-planning scenarios.

3. Data Sources and Preparation

IhingaAI separates historical model-development data, records entered into the application, and externally retrieved contextual information. This distinction is important because each source has a different scope, update frequency and evidentiary strength.

Data source	Use in IhingaAI	Interpretation
NISR 2025 SAS-derived prepared data	Development of frozen harvest-estimation models	Historical survey patterns, not live farm measurements.
Application database (SQLite)	Accounts, fields, saved estimates and operational records	Registered demo/application data, not representative national totals.
Open-Meteo	District-reference weather context	Forecasts approximate districts, not individual plots.
TangaBid / eSoko	Dated market-price information when available	Indicative records; not guaranteed sale prices.
Google Gemini	Optional explanatory text and tentative image guidance	Generated guidance; not a verified agronomic diagnosis.

3.1 Prepared datasets and model artifacts

The repository contains prepared modeling datasets under data/IhingaAI_Baseline_Model_Data.pkl and data/IhingaAI_Extension_Model_Data.pkl. These files are distinct from the two serialized trained model artifacts, data/IhingaAI_Frozen_Model_v1.pkl and data/IhingaAI_Frozen_Extension_v1.pkl. The project uses Git Large File Storage for the serialized files.

3.2 Data workflow

The project workflow includes reviewing source fields, checking missing or invalid values, preparing model inputs, separating development and evaluation data, fitting a supervised-learning estimator, and saving the fitted pipeline for inference. The packaged application loads the frozen artifacts, validates requests against model input rules and returns estimates only for supported cases.

3.3 Data governance

Use of survey-derived data must respect the original provider's authorization and redistribution terms. Public demonstration records should be synthetic. Personal contact details, secret keys and real farmer records should not be published in the repository.

4. Predictive Analytics Methodology

4.1 Estimation approach

IhingaAI provides crop-production estimates from historical agricultural survey patterns, using inputs such as crop, agricultural season, district, farmer category and field area where required by the relevant model. The baseline and extension artifacts are kept separate to preserve their model definitions and supported combinations. A frozen pipeline enables repeatable inference without retraining when a user requests an estimate.

4.2 Model outputs and safeguards

- Estimated production is reported in kilograms where a supported input combination is available.
- An accompanying range communicates uncertainty; it does not guarantee that a harvest will fall inside the interval.
- Unsupported crop-season combinations, invalid areas and out-of-scope inputs are flagged instead of being silently converted into plausible-looking forecasts.
- Field-level records and generative AI explanations do not retrain or overwrite the frozen predictive model.

4.3 Evidence and evaluation

Model development included a separate validation stage and error-based comparison against a crop-season median benchmark. Earlier recorded overall validation results for a log-target **Random Forest** were **MAE 1,997.688 kg, RMSE 61,681.521 kg and R^2 0.277 on 5,110 validation records; the benchmark MAE was 2,585.281 kg.** These are development-stage results, not an independent performance certification for all crops or current seasons. The large RMSE indicates sensitivity to extreme production observations. Performance must therefore be reviewed by crop, season and production scale before operational use.

4.4 Responsible use

Survey-derived prediction is a planning aid. It is not an agronomic prescription, insurance valuation, official production forecast or guarantee of profitability. On-field validation and expert review are needed before any consequential intervention.

5. Application Features and User Workflows

5.1 Farmer workspace

- Account access and field registration, including editing stored field attributes.
- Production estimation with supported-crop checks and saved estimate history.
- Field activities and task scheduling, plus crop-monitoring observations and an optional photo journal.
- Financial transaction records and field-related summaries for expense and income tracking.
- District weather and market information for contextual planning.
- AI guidance, field-risk screening and consultation options, subject to feature availability and network access.

5.2 Agronomist workspace

Agronomists can work with assigned and permission-shared farmer information, review relevant monitoring records and provide advisory notes. Farmer consent and assigned relationships constrain the intended scope of access. Recommendations support, rather than replace, direct assessment of farm conditions.

5.3 Planner workspace

The planner role supports district-oriented analytics, summaries of registered application records, agronomist assignment workflows and resource-scenario exploration. Any imported historical survey aggregates must remain visibly separate from user-entered data; district dashboards are not official NISR estimates.

5.4 End-to-end flow

A farmer signs in, registers a field, requests a supported crop-production estimate, records activities and observations, reviews supporting information and optionally seeks guidance. An authorized agronomist can follow up on shared records. A planner can inspect aggregated operational views and explore planning scenarios. This connected journey is the main integration offered by IhingaAI.

6. System Architecture and Technology

Layer	Technology or component	Responsibility
User interface	Python and Streamlit	Role-specific pages, inputs, summaries and navigation.
Application logic	Python modules in core/	Accounts, fields, prediction, activities, consultations, monitoring and reporting.
Predictive analytics	scikit-learn, pandas, NumPy	Load frozen pipelines and perform validated inference.
Local persistence	SQLite and local file storage	Demo accounts, field records and related operational data.
External services	Open-Meteo, market data, optional Google Gemini	Weather, dated price context and explanatory support.
Visualization	Plotly and Streamlit components	Charts, comparative views and dashboard controls.

6.1 Repository structure

- app.py: main Streamlit application entry point.
- core/: modular application services and role-related functionality.
- data/: crop and district reference files, prepared datasets, frozen model artifacts and labeled historical descriptives.
- assets/: logos, interface images and styling resources.
- Verification: Manual checks are recommended for authentication, role boundaries, model input validation and saving or retrieving records.
- requirements.txt, README.md and DATA_SOURCES.md: installation and attribution information.

6.2 Deployment architecture

The demonstration version runs through Streamlit, while the GitHub repository supports a local installation. A local SQLite database can be used to show prepared example records. Streamlit Community Cloud does not guarantee persistence of locally written SQLite files or uploaded photographs across application restarts. Cloud and local copies should not be assumed to share a synchronized database.

6.3 Access and secrets

Application roles are separated in the interface and relevant permission checks. External API credentials and the planner setup code should be provided through private deployment secrets rather than committed to source control. Additional security validation is required for production deployment.

7. Testing, Results and Limitations

7.1 Testing strategy

Recommended verification covers account access, cross-user data separation, supported and unsupported crop inputs, saved estimates, field records, role-specific navigation, farmer-agronomist sharing and external-service failures. Browser-level acceptance testing of login, refresh and logout remains important before the final demonstration.

7.2 Demonstrated outputs

The delivered prototype provides an integrated interface for field management and predictive insights, saved operational records, role-oriented workspaces and accessible contextual information. The presence of functional modules does not establish real-world crop-yield improvement or verified economic impact; neither should be claimed without field evaluation.

7.3 Known limitations

- Predictions are based on historical survey information and may be inaccurate for particular fields, crops or seasons.
- Certain combinations of inputs fall outside the validated model scope and must be rejected.
- District-reference weather may not match microclimates or real field conditions.
- Third-party market and AI services depend on connectivity, availability and configuration.
- Generative AI can produce incorrect or incomplete guidance and is not a substitute for agronomist diagnosis.
- Local SQLite storage is not appropriate as the sole persistent database for a production cloud service.
- Comprehensive security, usability, independent model validation and farmer-impact evaluations are still required.

7.4 Next development priorities

Priority improvements are persistent managed storage and backups, further testing of session/logout behavior, independent model assessment by crop and season, localized farmer usability trials, and formal review of agricultural recommendations by domain experts.

8. Installation, Demonstration and Submission Access

8.1 Run locally

1. Download or clone the GitHub repository, keeping the data/ folder and both frozen model artifacts available.

2. Install Python 3.11 or 3.12 and open a terminal in the project directory.

3. Run: python -m pip install -r requirements.txt

4. Start the application: python -m streamlit run app.py

5. Use the provided demonstration accounts where available. Configure optional integrations through private Streamlit secrets. A local demonstration database, if included, is separate from the cloud instance.

8.2 Access links

GitHub repository: https://github.com/Aime-Gentil/IhingaAI

Demo Video :
https://drive.google.com/file/d/1HkZN2P3bqKjj7SrVQSrFSIY3NqpEZ64u/view?usp=sharing

Live application: https://ihingaai-pioneers.streamlit.app

Login details:

User	Tel	Password
Farmer	0798935073	Aims@2005
Agronomist	0798935083	Aims@2005
Planner	0798935063	Aims@2005

9. Conclusion

IhingaAI demonstrates how agricultural survey-derived modeling can be combined with field management and advisory workflows in a single decision-support prototype. Its value lies in making estimates and contextual information more accessible to distinct agricultural users, while clearly distinguishing predictions, operational records and external services. Future deployment should prioritize validation, trustworthy data handling and reliable persistent infrastructure.

10. References and Project Resources

National Institute of Statistics of Rwanda (NISR). Seasonal Agricultural Survey 2025.

Open-Meteo. Weather forecast data and API documentation. https://open-meteo.com/

TangaBid / eSoko Rwanda. Agricultural market-price information. https://www.tangabid.com/market/prices/

Google Gemini. Optional AI service used for explanatory guidance. https://ai.google.dev/

IhingaAI project source, model artifacts and technical notes.

https://github.com/Aime-Gentil/IhingaAI